

TXSDK AT 指令开发指南



保密等级	A	TXSDK AT 指令开发指南	文件编号	TX-0000
发行日期	2024-05-07		文件版本	V1.0

责任与版权

责任限制

由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。

客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。

在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。

版权申明

泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。

未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK AT 指令开发指南	文件编号	TX-0000
发行日期	2024-05-07		文件版本	V1.0

修订记录

日期	版本	描 述	修订人
2024-05-07	V1.0		TX

保密等级	A	TXSDK AT 指令开发指南	文件编号	TX-0000
发行日期	2024-05-07		文件版本	V1.0

目录

TXSDK AT 指令开发指南 1

1. 概述1

2. AT 指令初始化2

3. 静态 AT 指令2

4. 动态 AT 指令3

5. 自定义 AT 指令3

6. 空指令4

7. Continue 模式4

1. 概述

TXSDK 是泰芯半导体发布的 WiFi/音视频系列芯片开发 SDK。本文档介绍了 TXSDK 的 AT 指令功能开发方法。

TXSDK AT 指令功能支持发送命令和数据，基本格式是：`at+xxx=a1,a2,a3`

AT 指令以字符名称作为唯一值，AT 指令名称不区分大小写。

AT 指令功能默认支持 16 个参数，以逗号作为分隔符。可以在初始化时修改分隔符和支持的参数数量。

SDK AT 指令功能默认对接到 `uart`，接收来自 `uart` 的数据。如有需要可以将 AT 指令功能对接到其他接口。

AT 指令功能支持转义字符和用引号包含参数。如果参数值本身包含引号、分隔符、转义字符，则需要使用使用转义字符标识出来，例如：

- `at+xxx=" arg data"` //引号包含参数
- `at+xxx=aaaa\",bbbb` //参数 1 的值包含了引号，使用转义字符。

AT 指令功能支持静态 AT 指令和动态 AT 指令。添加动态 AT 指令时会消耗内存资源，所以一般建议使用静态 AT 指令。

本文档只介绍 AT 指令开发方法，具体 AT 指令使用说明，请查看 AT 指令使用说明文档。

2. AT 指令初始化

SDK AT 指令功能初始化代码在 project/atcmd.c 文件的 sys_atcmd_init 函数，如下图所示：

```
__init void sys_atcmd_init(void)
{
    struct atcmd_settings setting;
    os_memset(&setting, 0, sizeof(setting));
    setting.args_count = ATCMD_ARGS_COUNT;
    setting.printbuf_size = ATCMD_PRINT_BUF_SIZE;
    setting.static_atcmds = static_atcmds;
    setting.static_cmdcnt = ARRAY_SIZE(static_atcmds);
    atcmd_uart_init(ATCMD_UARTDEV, 115200, 5, &setting);
}
```

上图的示例代码设置了 AT 指令参数数量，AT 指令打印输出的 buffer Size，静态 AT 指令列表，并对接到了 uart 接口。

初始化参数 `atcmd_settings` 介绍如下：

```
struct atcmd_settings{
    uint8  args_count, //AT 指令参数列表最大数量
           separator; //AT 指令参数的分隔符，默认是逗号
    uint16 static_cmdcnt; //静态 AT 指令列表的数量
    uint16 printbuf_size; //AT 指令使用的打印输出 buffer Size
    uint16 mute:1, //是否关闭 AT 指令的响应输出
           rev:15;
    const struct hgic_atcmd *static_atcmds; //静态 AT 指令列表
};
```

3. 静态 AT 指令

SDK 在 project/atcmd.c 中定义静态 AT 指令列表，使用静态 AT 指令的好处是占用的只是代码空间，节省了内存空间。

静态指令列表可以根据实际情况新增或删除部分指令。

4. 动态 AT 指令

SDK 支持添加动态 AT 指令，使用 `atcmd_register` API 注册动态指令，`atcmd_unregister` API 删除动态指令。

添加动态指令时需要分配内存空间，每个动态指令消耗约 16byte heap 空间。

```
int32 atcmd_register(const char *cmd, hgic_atcmd_hdl hdl)
```

- 参数 `cmd`: AT 指令名称
- 参数 `hdl`: AT 指令执行函数
- 返回值: 表示是否成功。0: 成功，非 0: 失败

```
int32 atcmd_unregister(const char *cmd)
```

- 参数 `cmd`: AT 指令名称
- 返回值: 表示是否成功。0: 成功，非 0: 失败

5. 自定义 AT 指令

新增 AT 指令可以使用静态 AT 指令，也可以使用动态 AT 指令。新增 AT 指令主要就是开发 AT 指令处理函数。AT 指令处理函数定义如下：

```
typedef int32(*hgic_atcmd_hdl)(const char *cmd, char *argv[], uint32 argc);
```

- 参数 `cmd`: 执行的 AT 指令名称
- 参数 `argv`: 该 AT 指令的参数列表，该参数是指针数组，`argv[0]`是第 1 个参数，`argv[1]`是第 2 个参数，...，`argv[argc-1]`
- 参数 `argc`: 该 AT 指令的参数个数
- 返回值: 表示 AT 指令处理结果，返回值为枚举 `ATCMD_RESULT`:

```
enum ATCMD_RESULT{  
    ATCMD_RESULT_ERR = -1, // 小于 0, atcmd 执行失败, 并返回错误码, 打印"ERROR: ret =错误码"  
    ATCMD_RESULT_OK = 0, // atcmd 执行成功并退出, 需要打印"OK"  
    ATCMD_RESULT_CONTINUE = 1, // atcmd 执行成功未退出, 进入 continue 模式  
    ATCMD_RESULT_DONE = 2, // atcmd 执行成功并退出, 不需要打印任何信息  
};
```

AT 指令模块在接收 AT 指令后，先根据指令名称查找对应的 AT 指令，然后以设置的

分隔符将参数分隔解析为参数列表，参数解析后执行 AT 指令的处理函数。根据 AT 指令处理函数的返回值，打印 ERROR，OK，或 进入 continue 模式。

6. 空指令

SDK AT 指令功能支持空指令。空指令是一种特殊的指令，在初始化 AT 指令时设置的 AT 指令名称为空字符串或 NULL 值。AT 指令模块接收 AT 指令数据后如果未查找到对应的 AT 指令，则执行空指令。

利用此特性，应用程序可以实现一些自定义功能。

SDK 只允许存在一个空指令。

7. Continue 模式

SDK AT 指令功能支持 continue 模式。continue 模式用于 AT 指令需要多次输入才完成一次操作的情况，例如使用 AT 指令发送多笔数据。

在开发自定义 AT 指令使用 continue 模式时，需要在 AT 指令的处理函数中自行识别当前 AT 指令是否已完成，如果未完成，则返回 ATCMD_RESULT_CONTINUE。返回值为 ATCMD_RESULT_CONTINUE 时 AT 指令功能对后续接收的数据不再执行参数解析，全部数据都作为该 AT 的数据，继续执行该指令。

进入 continue 模式后，输入的参数有所变化，此时 argv[0] 的值是接收的数据地址，argc 是数据长度。

退出 continue 模式，返回非 ATCMD_RESULT_CONTINUE 的值即可。可以根据实际情况返回 ATCMD_RESULT_DONE 或 ATCMD_RESULT_OK。